

TP3 – Sincronización de Tareas de FreeRTOS

Objetivos

Usar **Sincronización de Tareas** de **FreeRTOS**

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Editar/compilar/depurar programas en **C** con **FreeRTOS**

Configurar el **IDE** para depurar en modo **semihosting**

Adoptar metodología orientada a la portabilidad/reúso de **código**

Identificar **Patrones de Diseño de Software**

Usar **Git** y **GitHub**

Generar/actualizar **repositorio** por línea de **comando**

Generar/actualizar **archivos** del tipo **README.md** (**Markdown**)

Usar **Gemini**

Analizar/explicar **código fuente**

Sincronización de Tareas

Basic synchronization patterns: Rendezvous, **Mutual exclusion**, **Reusable barrier**.

Classical synchronization problems: **Producer - Consumer**, **Readers - Writers**.

Other problems: **Narrow vehicular bridge**, **Parking lot**, **Vehicular crossing**, **Security airlock**.

Actividades

1er encuentro

[TP3-01 – 12vo Proyecto p/placa NUCLEO-F103RB con FreeRTOS](#)

[TP3-02 – 13er Proyecto p/placa NUCLEO-F103RB con FreeRTOS](#)

2do encuentro

[TP3-03 – 14to Proyecto p/placa NUCLEO-F103RB con FreeRTOS](#)

[TP3-04 – 15to Proyecto p/placa NUCLEO-F103RB con FreeRTOS](#)

Resolución en grupos de **3** alumnos, requiere **presentación** por parte de un **responsable** del grupo

TP3 – Actividad 01 – 12vo Proyecto p/placa NUCLEO-F103RB con FreeRTOS

Objetivos

Usar Sincronización de Tareas de FreeRTOS

Crear/Gestionar/Sincronizar Tareas (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias) con **Semáforos binarios** (bloqueantes o no bloqueantes) y/o **Mutex**

Usar STM32CubeIDE

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar Git y GitHub

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Usar Gemini

Analizar/Explicar código fuente

Paso 01: Crear un nuevo repositorio en **GitHub** para almacenar su **TP3**, pasos:

GitHubRepositories => New =>

Repository name: **soe-tp3_****Año-Cuatrimestre_****Curso-Grupo**

Description: **# FIUBA - Electrónica - Sistemas Operativos Embebidos - Trabajo Práctico N°: 3 – Sincronización de Tareas de FreeRTOS**

Initialize this repository with:

Tildar **Add a README file**

=> Create repository

Año-Cuatrimestre es: **2026-1erC** o **2026-2doC** ... etc. (el correspondiente al Año y Cuatrimestre de cursada)
Curso-Grupo es: **1-01** o **1-02** ... etc. (el correspondiente al Curso y Grupo de cursada)

Paso 02: Invitar al repositorio, como **colaboradores** a los **docentes** de su curso, responsables de la aprobación de su **TP3**, pasos:

GitHub: Repositories => Settings => Collaborators => Tildar: Add people =>

en **Find people** => tipear: **"Username del Docente"**

Tildar: **Add "Username del Docente"**

Una vez que los **docentes** de su curso, **hayan aceptado** la **invitación** a ser **colaboradores**, podrá agregarlos como **Revisores**

PasPaso 03: Asentar los detalles de las entregas en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **editar** el archivo **README.md** y **modificar** su contenido, pasos:

GitHub: **READM.me => Edit this file => borrar** su contenido => **Copiar y Pegar** lo siguiente:

```
# FIUBA - Electrónica - Sistemas Operativos Embebidos
## Trabajo Práctico N°: 3 - Sincronización de Tareas de FreeRTOS
### Año-Cuatrimestre - Curso-Grupo

### Responsable de la entrega:
| Padrón | Apellidos, Nombres | Fecha | Deadline |
| :----- | :----- | :-----: | :-----: |
| XXXXXX | YYYY, ZZZ | | Semana 08 |
```

Actualizar el archivo **README.md** con los datos del responsable de la entrega (un alumno diferente para cada TP):

Año-Cuatrimestre, **Curso-Grupo**, **Padrón**, **Apellidos** y **Nombres**

Paso 04: Generar un nuevo **proyecto STM32**: **descargar** el archivo [soe-tp3_01-application.zip](#) e **importar** el **proyecto STM32** que contiene: [soe-tp3_01-application](#).

Compilar el **proyecto STM32** (**STM32CubeIDE**) importado: [soe-tp3_01-application](#).

Paso 05: Asentar los detalles de las entregas en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo [soe-tp3_01-application.md](#).

Paso 06: Consultar a **Gemini** de **Google** respecto al código fuente del **proyecto STM32**, pasos:

Logear, en **Google** con tu cuenta de correo electrónico institucional, del dominio **@fi.uba.ar**.

Ingresar a **Gemini** => <https://gemini.google.com/app>, en **Preguntar a Gemini** =>

Copiar y Pegar la siguiente línea de **texto**:

Analizar y explicar (en español), el funcionamiento del código fuente contenido en los archivos adjuntos: [app.c](#), [app_it.c](#), [task_a.c](#), [task_b.c](#) y [freertos.c](#).

Agregar los siguientes archivos (click en **+** Agregar archivos => **Subir archivos**):

```
...\soe_workspace\soe-tp3_01-application\app\src\app.c
...\soe_workspace\soe-tp3_01-application\app\src\app_it.c
...\soe_workspace\soe-tp3_01-application\app\src\task_a.c
...\soe_workspace\soe-tp3_01-application\app\src\task_b.c
...\soe_workspace\soe-tp3_01-application\app\src\freertos.c
```

Leer y almacenar la respuesta en: [...\soe_workspace\soe-tp3_01-application\soe-tp3_01-application.md](#).

Paso 07: **Depurar** el nuevo **proyecto STM32**.

Confirmar mediante la depuración, el análisis, las explicaciones e indicaciones de la respuesta de **Gemini Pro** del **Paso anterior**.

Paso 08: Almacenar el **proyecto STM32: soe-tp3_01-application** y el/los archivo/s de la actividad: **soe-tp3_01-application.md**, en el **repositorio** creado en **GitHub**, en el **TP3 – Actividad 01 – Paso 01**.

Paso 09: **Modificar** los mecanismos de sincronización entre las **task_a** y **task_b**, recurrir a **Semáforos binarios** y/o **Mutex** (bloqueante o no bloqueante, excluyente, lo adecuado), para implementar el **Classical synchronization problems: Producer - Consumer**, **compilar/depurar/observar** comportamiento, **asentar** lo observado en el archivo **soe-tp3_01-application.md**.

Referencias:

- **The Little Book of Semaphores**
 - [Download The Little Book of Semaphores in PDF](#) (Allen B. Downey - Version 2.2.1)
 - **Chapter 4 Classical synchronization problems**
 - **4.1 Producer-consumer problem**

Paso 10: Almacenar el **proyecto STM32: soe-tp3_01-application** y el/los archivo/s de la actividad: **soe-tp3_01-application.md**, en el **repositorio** creado en **GitHub**, en el **TP3 – Actividad 01 – Paso 01**.

TP3 – Actividad 02 – 13er Proyecto p/placa NUCLEO-F103RB con FreeRTOS

Objetivos

Usar **Sincronización de Tareas de FreeRTOS**

Crear/Gestionar/Sincronizar Tareas (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias) con **Semáforos binarios** (bloqueantes o no bloqueantes) y/o **Mutex**

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Usar **Gemini**

Analizar/Explicar código fuente

Paso 01: **Generar** un nuevo **proyecto STM32**: **descargar** el archivo [soe-tp3_02-application.zip](#) e **importar** el **proyecto STM32** que contiene: [soe-tp3_02-application](#).

Compilar el **proyecto STM32** (**STM32CubeIDE**) importado: [soe-tp3_02-application](#).

Paso 02: **Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo [soe-tp3_02-application.md](#).

Paso 03: **Consultar** a **Gemini** de **Google** respecto al código fuente del **proyecto STM32**, pasos:

Ingresar a **Gemini** => <https://gemini.google.com/app>, en **Preguntar a Gemini** =>

Copiar y Pegar la siguiente línea de **texto**:

Analizar y explicar (en español), el funcionamiento del código fuente contenido en los archivos adjuntos: [app.c](#), [app_it.c](#), [task_a.c](#), [task_b.c](#), y [freertos.c](#).

Agregar los siguientes archivos (click en **+** Agregar archivos => **Subir archivos**):

```
...\soe_workspace\soe-tp3_02-application\app\src\app.c  
...\soe_workspace\soe-tp3_02-application\app\src\app_it.c  
...\soe_workspace\soe-tp3_02-application\app\src\task_a.c  
...\soe_workspace\soe-tp3_02-application\app\src\task_b.c  
...\soe_workspace\soe-tp3_02-application\app\src\freertos.c
```

Leer y almacenar la respuesta en: [...\soe_workspace\soe-tp3_02-application\soe-tp3_02-application.md](#).

Paso 04: **Depurar** el nuevo **proyecto STM32**.

Confirmar mediante la depuración, el análisis, las explicaciones e indicaciones de la respuesta de **Gemini Pro** del **Paso anterior**.

Paso 05: Almacenar el **proyecto STM32: soe-tp3_02-application** y el/los archivo/s de la actividad: **soe-tp3_2-application.md**, en el **repositorio** creado en **GitHub**, en el **TP3 – Actividad 01 – Paso 01**.

Paso 06: **Modificar** los mecanismos de sincronización entre las **task_a** y **task_b**, recurrir a **Semáforos binarios** y/o **Mutex** (bloqueante o no bloqueante, excluyente, lo adecuado), para implementar el **Classical synchronization problems: Readers - Writers**, **compilar/depurar/observar** comportamiento, **asentar** lo observado en el archivo **soe-tp3_02-application.md**.

Referencia:

- **The Little Book of Semaphores**
 - [Download The Little Book of Semaphores in PDF](#) (Allen B. Downey - Version 2.2.1)
 - **Chapter 4 Classical synchronization problems**
 - **4.2 Readers-writers problem**

Paso 07: Almacenar el **proyecto STM32: soe-tp3_02-application** y el/los archivo/s de la actividad: **soe-tp3_02-application.md**, en el **repositorio** creado en **GitHub**, en el **TP3 – Actividad 01 – Paso 01**.

TP3 – Actividad 03 – 14to Proyecto p/placa NUCLEO-F103RB con FreeRTOS

Objetivos

Usar Sincronización de Tareas de FreeRTOS

Crear/Gestionar/Sincronizar **Tareas** (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias) con **Semáforos binarios** (bloqueantes o no bloqueantes) y/o **Mutex**

Usar STM32CubeIDE

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar Git y GitHub

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Usar Gemini

Analizar/Explicar código fuente

Paso 01: Generar un nuevo **proyecto STM32**: descargar el archivo [soe-tp3_03-application.zip](#) e importar el **proyecto STM32** que contiene: [soe-tp3_03-application](#).

Compilar el **proyecto STM32** (**STM32CubeIDE**) importado: [soe-tp3_03-application](#).

Paso 02: Asentar los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo [soe-tp3_03-application.md](#).

Paso 03: Consultar a **Gemini** de **Google** respecto al código fuente del **proyecto STM32**, pasos:

Ingresar a **Gemini** => <https://gemini.google.com/app>, en **Preguntar a Gemini** =>

Copiar y Pegar la siguiente línea de **texto**:

Analizar y explicar (en español), el funcionamiento del código fuente contenido en los archivos adjuntos: [app.c](#), [app_it.c](#), [task_enrtry_a.c](#), [task_exit_a.c](#), [task_test.c](#) y [freertos.c](#).

Agregar los siguientes archivos (click en **+** Agregar archivos => **Subir archivos**):

```
...\soe_workspace\soe-tp3_03-application\app\src\app.c
...\soe_workspace\soe-tp3_03-application\app\src\app_it.c
...\soe_workspace\soe-tp3_03-application\app\src\task_entry_a.c
...\soe_workspace\soe-tp3_03-application\app\src\task_exit_a.c
...\soe_workspace\soe-tp3_03-application\app\src\task_test.c
...\soe_workspace\soe-tp3_03-application\app\src\freertos.c
```

Leer y almacenar la respuesta en: [...\soe_workspace\soe-tp3_03-application\soe-tp3_03-application.md](#).

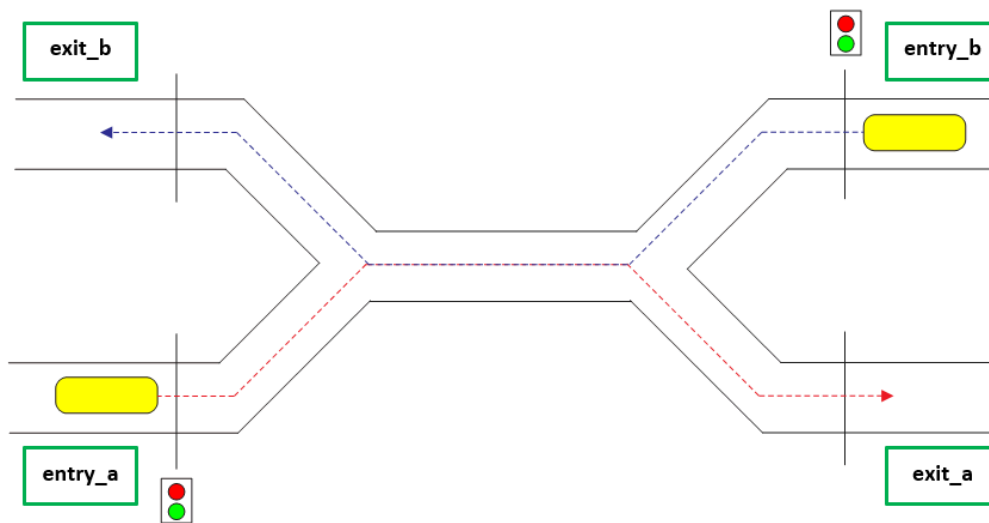
Paso 04: Depurar el nuevo **proyecto STM32**.

Confirmar mediante la depuración, el análisis, las explicaciones e indicaciones de la respuesta de **Gemini Pro** del **Paso anterior**.

Paso 05: Almacenar el **proyecto STM32: soe-tp3_03-application** y el/los archivo/s de la actividad: **soe-tp3_3-application.md**, en el **repositorio** creado en **GitHub**, en el **TP3 – Actividad 01 – Paso 01**.

Paso 06: **Modificar** los mecanismos de sincronización entre las **task_test**, **task_entry_a** y **task_exit_a**, recurrir a **Semáforos binarios** y/o **Mutex** (bloqueante o no bloqueante, excluyente, lo adecuado), para implementar el **Other problems: Vehicular crossing**, **compilar/depurar/observar** comportamiento, **asentar** lo observado en el archivo **soe-tp3_03-application.md**.

Implementar un Sistema de **Control de Acceso** que **monitorea y controla** tanto el **ingreso/egreso** de **vehículos** (**dos** puntos de **ingreso** y **dos** puntos de **egreso**) a un **cruce vial** de **capacidad** limitada.



Para la implementación se sugieren **cuatro tareas** (**dos** puntos de **ingreso** y **dos** puntos de **egreso**):

```
void task_entry_a(void *parameters); /* Monitoreo y Control del ingreso de vehículos */
void task_exit_a(void *parameters); /* Monitoreo y Control del egreso de vehículos */

void task_entry_b(void *parameters); /* Monitoreo y Control del ingreso de vehículos */
void task_exit_b(void *parameters); /* Monitoreo y Control del egreso de vehículos */
```

Observaciones:

- Al **cruce vial** los **vehículos**, **ingresan/egresan** por **orden** de llegada hasta colmar la **capacidad** del mismo.
- La capacidad del **cruce vial** está limitada a **G_TASKS_CNT_MAX** vehículos.
- Cada uno de los puntos de **ingreso** al cruce vial cuenta con un **semáforo vial** (**Rojo**, **Verde**) a controlar.
- Una quinta **tarea** de **test** (periódicamente recupera un estímulo de un array y lo envía las otras tareas):

```
void task_test (void *parameters);
```

Estímulos: **ENTRY_A**, **EXIT_A**, **ENTRY_B**, **EXIT_B**.

Pasos recomendados:

- Agregar a `task_test()` y `task_entry_a()`, la sincronización correspondiente al **ingreso** de **vehículos** (**semáforo binario**).
- Agregar a `task_test()` y `task_exit_a()`, la sincronización correspondiente al **egreso** de **vehículos** (**semáforo binario**).
- Agregar a `task_entry_a()` y `task_exit_a()`, la sincronización correspondiente a la **detección** de límite (`G_TASKS_CNT_MAX`) de **capacidad** e **impedir** el **ingreso** en tal condición (**contador** + **mutex**).
- Agregar a `task_entry_a()` y `task_exit_a()`, el control del **semáforo vial** (**Rojo**, **Verde**).
- Repetir el procedimiento para `task_entry_b()` y `task_exit_b()`.
- Agregar a `task_entry_a()` y `task_entry_b()`, la sincronización correspondiente al **acceso** a **recursos compartidos** por las **tareas** de **monitoreo** y **control** de **ingreso** de **vehículos** (**mutex**).

Paso 07: Almacenar el **proyecto STM32: soe-tp3_03-application** y el/los archivo/s de la actividad: **soe-tp3_03-application.md**, en el **repositorio** creado en **GitHub**, en el **TP3 – Actividad 01 – Paso 01**.

TP3 – Actividad 04 – 15to Proyecto p/placa NUCLEO-F103RB con FreeRTOS

Objetivos

Usar **Sincronización de Tareas de FreeRTOS**

Crear/Gestionar/Sincronizar Tareas (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias) con **Semáforos binarios** (bloqueantes o no bloqueantes) y/o **Mutex**

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Usar **Gemini**

Analizar/Explicar código fuente

Paso 01: **Generar** un nuevo **proyecto STM32**: **descargar** el archivo [soe-tp3_04-application.zip](#) e **importar** el **proyecto STM32** que contiene: [soe-tp3_04-application](#).

Compilar el **proyecto STM32** (**STM32CubeIDE**) importado: [soe-tp3_04-application](#).

Paso 02: **Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo [soe-tp3_04-application.md](#).

Paso 03: **Consultar** a **Gemini** de **Google** respecto al código fuente del **proyecto STM32**, pasos:

Ingresar a **Gemini** => <https://gemini.google.com/app>, en **Preguntar a Gemini** =>

Copiar y Pegar la siguiente línea de **texto**:

Analizar y explicar (en español), el funcionamiento del código fuente contenido en los archivos adjuntos: [app.c](#), [app_it.c](#), [task_enrtry_a.c](#), [task_exit_a.c](#), [task_test.c](#) y [freertos.c](#).

Agregar los siguientes archivos (click en **+** Agregar archivos => **Subir archivos**):

```
...\soe_workspace\soe-tp3_04-application\app\src\app.c
...\soe_workspace\soe-tp3_04-application\app\src\app_it.c
...\soe_workspace\soe-tp3_04-application\app\src\task_entry_a.c
...\soe_workspace\soe-tp3_04-application\app\src\task_exit_a.c
...\soe_workspace\soe-tp3_04-application\app\src\task_test.c
...\soe_workspace\soe-tp3_03-application\app\src\freertos.c
```

Leer y almacenar la respuesta en: [...\soe_workspace\soe-tp3_04-application\soe-tp3_04-application.md](#).

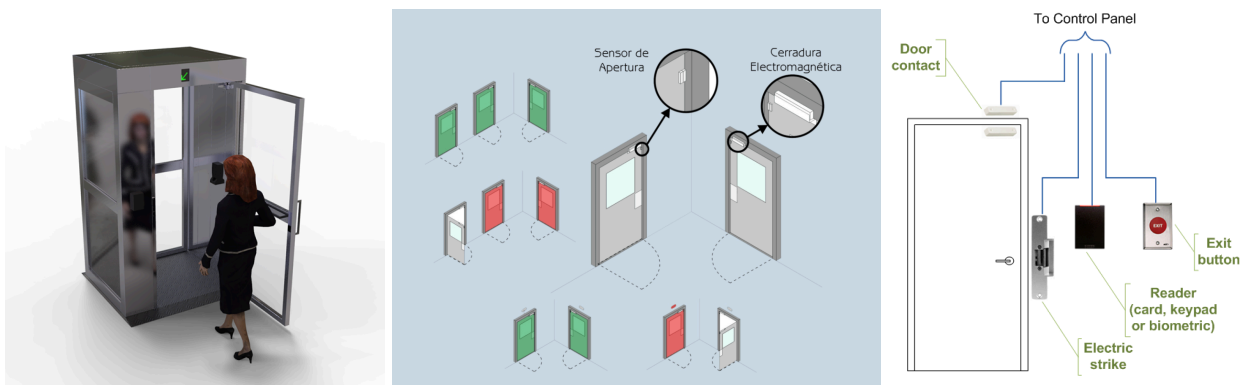
Paso 04: **Depurar** el nuevo **proyecto STM32**.

Confirmar mediante la depuración, el análisis, las explicaciones e indicaciones de la respuesta de **Gemini Pro** del **Paso anterior**.

Paso 05: Almacenar el **proyecto STM32: soe-tp3_04-application** y el/los archivo/s de la actividad: **soe-tp3_4-application.md**, en el **repositorio** creado en **GitHub**, en el **TP3 – Actividad 01 – Paso 01**.

Paso 06: **Modificar** los mecanismos de sincronización entre las **task_test**, **task_entry_a** y **task_exit_a**, recurrir a **Semáforos binarios** y/o **Mutex** (bloqueante o no bloqueante, excluyente, lo adecuado), para implementar el **Other problems: Security airlock**, **compilar/depurar/observar** comportamiento, **asentar** lo observado en el archivo **soe-tp3_04-application.md**.

Implementar un Sistema de **Control de Acceso** del tipo **puerta esclusa** que **monitorea y controla** tanto el **ingreso/egreso** de **personas** (**cuatro** puntos de **ingreso/egreso**) de **capacidad** limitada (sólo una persona a la vez).



Para la implementación se sugieren **cuatro tareas** (**cuatro** puntos de **ingreso/egreso**):

```
void task_gate_a(void *parameters); /* Monitoreo y Control del ingreso/egreso de personas*/
void task_gate_b(void *parameters); /* Monitoreo y Control del ingreso/egreso de personas*/
void task_gate_c(void *parameters); /* Monitoreo y Control del ingreso/egreso de personas*/
void task_gate_d(void *parameters); /* Monitoreo y Control del ingreso/egreso de personas*/
```

Observaciones:

- A la **puerta esclusa** las **personas**, **ingresan/egresan** por **orden** de llegada, permitiendo el ingreso de **una persona** a la vez, normalmente las **puertas** están **cerradas** y el sistema permite **abrir una puerta** a la vez (ver imagen de **puertas coloreadas**).
- Cada uno de los puntos de **ingreso/egreso** a la **puerta esclusa** cuenta con un **control de puerta** (**Close, Open**) para controlar la **apertura/cierre** de la **puerta**.
- Una cuarta **tarea** de **test** (periódicamente recupera un estímulo de un array y lo envía las otras tareas):

```
void task_test (void *parameters);
```

Estímulos: **OPEN_REQUEST_A, DOOR_CLOSED_A, OPEN_REQUEST_B, DOOR_CLOSED_B, OPEN_REQUEST_C, DOOR_CLOSED_C, OPEN_REQUEST_D, DOOR_CLOSED_D**.

Pasos recomendados:

- Agregar a **task_test()** y **task_gate_a()**, la sincronización correspondiente a **pedido de apertura** y a **puerta cerrada**.
- Idem anterior para **task_gate_b()**.

- Agregar a `task_gate_a()` y `task_gate_b()`, la sincronización correspondiente al **acceso** a **recursos compartidos** por las **tareas** de **monitoreo** y **control** de **ingreso/egreso** de **personas** (**mutex**).
- Agregar a `task_gate_a()` y `task_gate_b()`, el **control de puerta** (**Close**, **Open**) para controlar la **apertura/cierre** de la **puerta**.
- Repetir el procedimiento para `task_entry_c()` y `task_exit_d()`.

Paso 07: Almacenar el **proyecto STM32: soe-tp3_04-application** y el/los archivo/s de la actividad: **soe-tp3_04-application.md**, en el **repositorio** creado en **GitHub**, en el **TP3 – Actividad 01 – Paso 01**.

GitHub: Presionar: **Commit changes ... (Guardar los cambios)**

Tildar: **Create a new branch** for this commit and start a pull request

Presionar: **Propose changes**

Presionar: **Create pull request**, tipear el nombre del **branch** y **agregar** a los **docentes** de su curso, como **Reviewers** (para invitarlos a **revisar** sus **entregas** mediante un **pull request**)

Ver **cómo hacer el pull request** en este video: [Trabajo colaborativo con Github](#)

En la descripción del video se detalla el contenido (ver en [14:23](#) la sección "**CÓMO SOLICITAR REVISIONES**")
IMPORTANTE: Se recomienda que primero vean "[19:25](#) Solución a un problema muy habitual" y luego "[19:02](#) Create pull request y Review requested"